==========

==========

CHAPTER 7 — STAGE-2 TRANSLATION (IPA → PA)

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of how ARMv8 performs Stage-2 translation, including hypervisor-controlled permissions, memory attributes, VM isolation, and final physical address generation.

****

****

7.1 — Introduction to Stage-2 Translation

****

****

Stage-2 translation is performed by the **hypervisor (EL2)**. Its purpose is to convert:

```
Intermediate Physical Address (IPA) → Physical Address (PA)
```

Stage-2 defines:

- VM isolation
- hypervisor-controlled permissions
- hypervisor-controlled memory attributes
- device memory mapping
- real physical memory layout
- security boundaries between VMs

Stage-2 is **authoritative**. It can override any Stage-1 decision.

****

****

7.2 — Stage-2 Translation Regime

****

****

Stage-2 applies to:

- EL0/EL1 accesses inside a VM
- EL2 accesses when HCR_EL2.TGE = 0
- IPA accesses generated by Stage-1

Stage-2 is controlled by:

- **TTBR0_EL2** (root of Stage-2 tables)
- **TCR_EL2** (translation configuration)
- **MAIR_EL2** (memory attributes)
- **HCR_EL2** (hypervisor control)

Teaching Note

Stage-2 is mandatory for virtualization. Without Stage-2, VMs cannot be isolated.

****

****

7.3 — Stage-2 Translation Flow

****

****

High-level flow:

```
IPA
 │
 ▼
Stage-2 TLB lookup
 │
 ├── Hit → PA
 ▼
Stage-2 page table walk
 │
 ▼
Descriptor interpretation
 │
 ▼
PA generation
 │
 ▼
Insert into TLB
```

****

****

7.4 — TTBR0_EL2 (Stage-2 Root)

****

****

TTBR0_EL2 defines the root of Stage-2 translation tables.

Contains:

- physical base address
- memory attribute overrides
- shareability overrides

Implementation Note

TTBR0_EL2 must be configured before enabling Stage-2 MMU.

****

****

7.5 — Stage-2 TLB Lookup

****

****

TLB entries are tagged with:

- IPA
- PA
- VMID
- permissions
- attributes

TLB lookup requires:

```
IPA + VMID match
```

If hit:

- permissions checked
- attributes applied
- PA returned

If miss:

- Stage-2 page table walk begins.

****

****

7.6 — Stage-2 Page Table Walk

****

****

Walk steps:

1. Read L1 descriptor
2. If table → read L2
3. If table → read L3
4. If page/block → final mapping

5. Extract fields
6. Produce PA

Walk cache accelerates steps 1–3.

Teaching Note

Stage-2 walk is identical to Stage-1 walk, but uses MAIR_EL2.

****

****

7.7 — Stage-2 Descriptor Interpretation

****

****

Stage-2 descriptors define:

- physical base address (PA base)
- S2AP (access permissions)
- S2SH (shareability)
- S2AttrIndx (memory attributes)
- S2PXN/S2UXN (execute permissions)
- AF (access flag)

Implementation Note

Stage-2 descriptors override Stage-1 descriptors.

****

****

7.8 — Stage-2 Access Flag (AF)

****

****

AF indicates whether the page has been accessed.

- **AF = 1** → page is accessible
- **AF = 0** → Access Flag Fault

Hypervisor may use AF for:

- dirty page tracking
- live migration
- memory compression
- page aging

****

****

7.9 — Stage-2 Access Permissions (S2AP)

****

****

S2AP controls read/write permissions.

S2AP	Meaning
00	Read/write
01	Read-only
10	Reserved
11	No access

Teaching Note

S2AP overrides Stage-1 AP.

****

****

7.10 — Stage-2 Execute Permissions (S2PXN/S2UXN)

****

****

S2PXN → EL1 cannot execute S2UXN → EL0 cannot execute

Hypervisor can enforce:

- non-exec guest kernel
- non-exec guest user space
- W^X policy
- code integrity

Implementation Note

Hypervisor can force non-exec even if guest marks executable.

****

****

7.11 — Stage-2 Shareability (S2SH)

****

****

S2SH controls cache coherence.

S2SH	Meaning
00	Non-shareable
10	Outer Shareable
11	Inner Shareable

Teaching Note

Hypervisor can override guest shareability.

****

****

7.12 — Stage-2 Memory Attributes (S2AttrIndx → MAIR_EL2)

****

****

AttrIndx selects MAIR_EL2 entry.

Hypervisor uses MAIR_EL2 to enforce:

- device memory
- cacheability
- ordering rules

Example

Guest marks MMIO as Normal WB. Hypervisor forces Device-nGnRnE.

****

****

7.13 — PA Generation

****

****

PA is computed as:

```
PA = PA_base + page_offset
```

Where:

- PA_base = descriptor[47:12]
- page_offset = IPA[11:0]

Teaching Note

PA is the real physical address used by hardware.

****

****

7.14 — Stage-2 Permission Faults

****

****

Faults include:

- Access Flag Fault
- Permission Fault
- Execute Fault
- Alignment Fault
- Translation Fault

Implementation Note

Stage-2 faults are delivered to the hypervisor.

****

****

7.15 — Stage-2 Translation Example (Full Walkthrough)

****

****

Given:

`IPA = 0x0000_2000_1234_5678`

Step 1 — TLB lookup

Miss.

Step 2 — L1 descriptor

Type = table → go to L2.

Step 3 — L2 descriptor

Type = block → final mapping.

Step 4 — Extract fields

- PA_base = 0x4000_0000_0000_0000
- S2AP = read-only
- S2SH = Inner Shareable
- S2AttrIndx = 1 (Device-nGnRnE)
- S2PXN = 1

Step 5 — PA generation

PA = PA_base + offset.

****

****

7.16 — Stage-2 and VM Isolation

****

****

VMID identifies:

- which VM owns a TLB entry
- which Stage-2 mappings are valid
- which entries must be invalidated on VM switch

Teaching Note

VMID + ASID provides full isolation:

```
VM isolation + process isolation
```

****

****

7.17 — Stage-2 and Nested Virtualization

****

****

Hypervisor may emulate Stage-2 for nested VMs.

Requires:

- shadow page tables
- nested TLB invalidation
- nested VMID allocation

Implementation Note

Nested virtualization is complex and performance-sensitive.

****

****

7.18 — Stage-2 and Device Mapping

****

****

Hypervisor maps:

- MMIO
- PCIe BARs
- interrupt controllers
- DMA buffers
- firmware regions

Device memory must be:

- Device-nGnRnE
- non-cacheable
- strongly ordered

Teaching Note

Incorrect device attributes cause hardware malfunction.

****

****

7.19 — Stage-2 and Memory Overcommit

****

****

Hypervisor may:

- map same PA to multiple IPAs
- remap IPA to different PA
- swap pages to disk
- balloon memory
- compress pages

Implementation Note

Stage-2 enables advanced VM memory management.

****

****

7.20 — Stage-2 and Live Migration

****

****

Hypervisor uses Stage-2 to:

- track dirty pages
- remap pages
- freeze guest memory
- copy memory to destination host

AF and S2AP are used for dirty tracking.

****

****

7.21 — Stage-2 and Security

****

****

Hypervisor enforces:

- VM isolation
- non-exec pages
- read-only pages
- device memory protection
- kernel integrity
- guest sandboxing

Teaching Note

Stage-2 is the primary security boundary in virtualization.

****

****

7.22 — Stage-2 Fault Injection

****

****

Hypervisor may intentionally generate faults for:

- memory introspection
- debugging
- sandboxing
- security monitoring

****

****

7.23 — Stage-2 and QNX/RTOS Behavior

****

****

QNX uses Stage-2 for:

- partitioned VMs
- safety-critical isolation
- deterministic memory mapping

- device partitioning

Implementation Note

QNX may use static VMID ranges.

****

****

7.24 — Chapter Summary

****

****

This chapter covered:

- Stage-2 translation flow
- TTBR0_EL2
- TLB lookup
- page table walk
- descriptor interpretation
- S2AP, S2PXN/S2UXN, S2SH, S2AttrIndx
- PA generation
- VMID isolation
- nested virtualization
- device mapping
- memory overcommit
- live migration
- security
- QNX/RTOS considerations